Object-Oriented Programming – Project Work:

SPACE COLONY

The purpose of this project is to build an application where the player manages a space colony crew. The program features five different crew member specializations (Pilot, Engineer, Medic, Scientist, and Soldier). The user can recruit crew members, train them in a simulator, and send them on cooperative missions against system-generated threats.

The application will be developed as an Android app, where the user can input all necessary values through a graphical user interface and command the crew members directly from menus.

NOTE: The instructions given in this document are not the only way to implement the Space Colony game. You can do something similar and use your judgment on how to do the actual implementation.

The project is to be completed in groups of 1–3 students.

IMPORTANT: There are no resubmissions for this project. You have only one attempt to submit the project, and no fixes or corrections are allowed after submission. Make sure your submission is complete and tested before the deadline.

There are two submissions in the project work: 1) Project plan and 2) Final project submission.

Project Plan

The project plan is a short but concise plan on what you are implementing (the topic), what features it will contain, and who is on your team. The plan must include a UML diagram of your program and a preliminary user interface description.

Also write what features you plan on implementing (include the bonus points). You can change your mind later, this is just for us teaching staff to prepare for validation.

The deadline for submitting the project plan is March 30, 2026, at 23:59 (Finnish time).

Final Project Submission

Return

1) a link to the project GitHub (that is accessible by the teacher! So no private repository links!),
2) along with a video describing the program,
3) the project documentation
    a. Format is free, but must describe the project, how it was implemented, UML diagrams describing the implementation, application use-flow, installation instructions (if needed), where GitHub and Video links are to be found, team composition, how was work shared, tools used. Also include the listing of bonus features that you implemented!
4) and your team composition.
5) Disclaimer on AI usage (if AI was used, where and how)

If you miss any of these five items, your submission is automatically graded as failed. No second submission chances are given.

The deadline for the final project submission is April 20, 2026, at 23:59 (Finnish time).

1 Instructions

The program allows the creation of crew members, which are initially placed in "Quarters" (home). The user can then move them to the Simulator (training area) or Mission Control (battle arena).

When a crew member is trained, they gain experience points that affect their mission performance. For example, if a crew member has 2 experience points, their skill power increases by 2 points.

The mission system is cooperative and turn-based. Two crew members are sent on a mission together against a system-generated threat. The crew members take turns acting against the threat, and the threat retaliates after each action. The mission continues until either the threat is neutralized or both crew members' energy drops to zero. If the threat is defeated, surviving crew members earn experience points. A crew member whose energy drops to zero dies and is removed from the program.

Threats can be whatever you want. Asteroid storm, broken heating, alien attack, solar flares, fuel leakage, fire in the kitchen – you name it ☺

Once a crew member is returned to Quarters, they fully regenerate their energy but retain their accumulated experience points.

1.1 Mission Algorithm

A possible algorithm for handling missions could be as follows:

1. Allow the user to select two crew members from those available in Mission Control.
2. Generate a threat with stats based on a scaling formula (e.g., threat skill = 4 + number of completed missions).
3. Enter a while loop to execute the mission.
    a. The mission can be automatic or turn-based, where the player chooses actions. You can decide how to do it yourself. Extra points will be awarded if the player can choose different actions per turn.
4. Print the stats of both crew members (A and B) and the Threat.
5. Execute the action: example: *Threat.defend(A.act());*
6. If Threat is still active, it retaliates: example: *A.defend(Threat.act());*
    b. If A survives, swap to crew member B for the next turn.
    c. If A's energy drops to zero, A is removed. B continues alone.
7. If the Threat's energy reaches zero:
    a. Print the message "The threat has been neutralized!"
    b. Award experience points to surviving crew members.
    c. Return surviving crew members to Mission Control.
    d. Exit the loop.
8. If both crew members' energy reaches zero:
    d. Print the message "Mission failed. All crew members lost."
    e. Remove both crew members from the program.

Table 1 presents an example of default values for different crew members. You can modify and change these as you see fit; this is just a suggestion.

Table 1: Example of Default Values for Crew Members Upon Recruitment

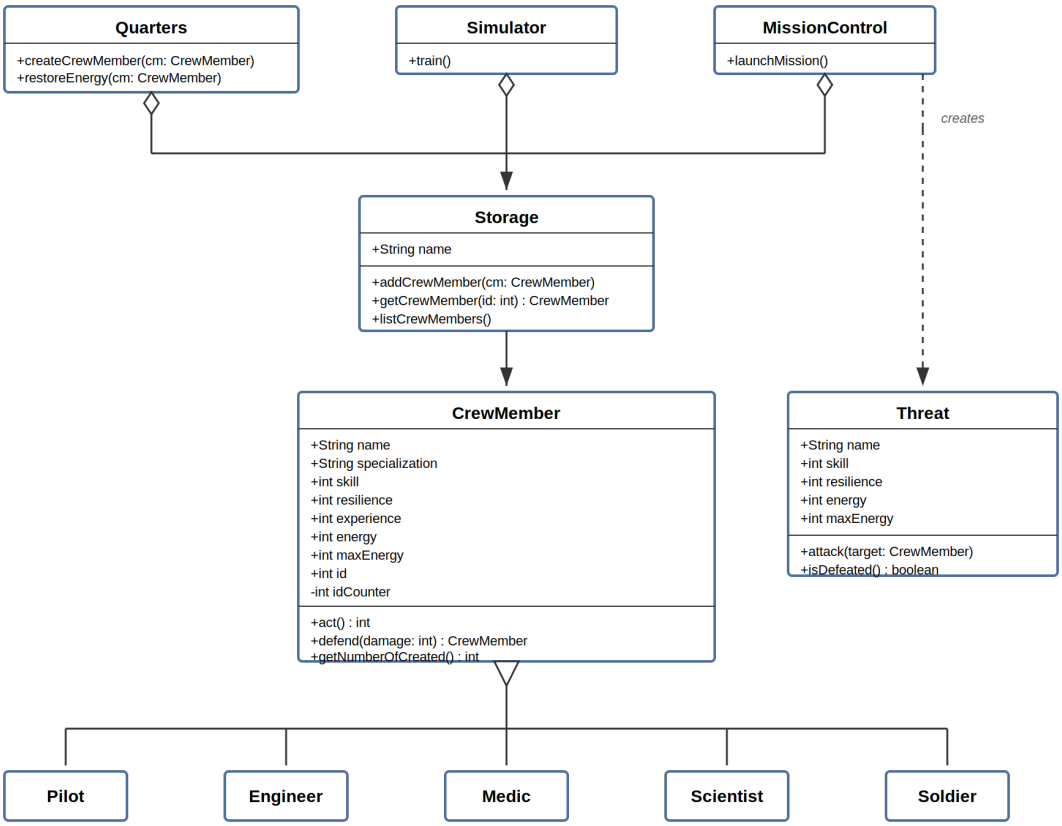
Specialization	Skill	Resilience	Max Energy	Color
Pilot	5	4	20	Blue
Engineer	6	3	19	Yellow
Medic	7	2	18	Green
Scientist	8	1	17	Purple
Soldier	9	0	16	Red

Experience starts at zero, and energy begins at its maximum value.

1.2 Class Diagram (example)

The class diagram could be something like the one shown here. However, it can differ depending on your own implementation.

The Threat class represents system-generated enemies that crew members fight cooperatively. MissionControl creates Threat instances for each mission.



1.3 Data Structures

You can choose which data structures to use, but in the Storage class, a HashMap can be quite handy since it can store both the crew member and its ID:

```
HashMap<Integer, CrewMember>
```

With an ArrayList, you may need to implement various loops to manage the data, but handling it in RecyclerView is more straightforward.

For threat generation, you can maintain a mission counter (static or in MissionControl) that determines the difficulty scaling of generated threats.

1.4 Example of a Cooperative Mission

Below is an example of how a cooperative mission might play out:

```
=== MISSION: Asteroid Field Navigation ===
Threat: Asteroid Storm (skill: 6, resilience: 2, energy: 25/25)

Crew Member A: Pilot(Nova) skill: 5; res: 4; exp: 0; energy: 20/20
Crew Member B: Soldier(Rex) skill: 9; res: 0; exp: 0; energy: 16/16

--- Round 1 ---
Pilot(Nova) acts against Asteroid Storm
  Damage dealt: 5 - 2 = 3
  Asteroid Storm energy: 22/25
Asteroid Storm retaliates against Pilot(Nova)
  Damage dealt: 6 - 4 = 2
  Pilot(Nova) energy: 18/20

Soldier(Rex) acts against Asteroid Storm
  Damage dealt: 9 - 2 = 7
  Asteroid Storm energy: 15/25
Asteroid Storm retaliates against Soldier(Rex)
  Damage dealt: 6 - 0 = 6
  Soldier(Rex) energy: 10/16

--- Round 2 ---
Pilot(Nova) acts against Asteroid Storm
  Damage dealt: 5 - 2 = 3
  Asteroid Storm energy: 12/25
Asteroid Storm retaliates against Pilot(Nova)
  Damage dealt: 6 - 4 = 2
  Pilot(Nova) energy: 16/20

Soldier(Rex) acts against Asteroid Storm
  Damage dealt: 9 - 2 = 7
  Asteroid Storm energy: 5/25
Asteroid Storm retaliates against Soldier(Rex)
  Damage dealt: 6 - 0 = 6
  Soldier(Rex) energy: 4/16

--- Round 3 ---
Pilot(Nova) acts against Asteroid Storm
  Damage dealt: 5 - 2 = 3
  Asteroid Storm energy: 2/25
Asteroid Storm retaliates against Pilot(Nova)
  Damage dealt: 6 - 4 = 2
  Pilot(Nova) energy: 14/20

Soldier(Rex) acts against Asteroid Storm
  Damage dealt: 9 - 2 = 7
  Asteroid Storm energy: 0/25

=== MISSION COMPLETE ===
The Asteroid Storm has been neutralized!
Pilot(Nova) gains 1 experience point. (exp: 1)
Soldier(Rex) gains 1 experience point. (exp: 1)
```

1.5 UI Mockups

Here are some ideas for the user interfaces. Feel free to implement them however you like. This is just to give some ideas to get started with.

Home Screen / Colony Overview

- Shows summary of crew in each location (Quarters, Simulator, Mission Control)
- Buttons to navigate to each location and to create new crew members

Recruit Crew Member Screen

- Text input for crew member name
- Radio buttons or dropdown to select specialization (with stats shown)
- Create and Cancel buttons

Quarters Screen

- List of crew members at home with their stats
- Checkboxes to select crew members
- Buttons: "Move to Simulator" and "Move to Mission Control"

Simulator Screen

- List of crew members in training
- "Train Selected" button that awards experience
- "Move to Quarters" button to send crew home (restores energy)

Mission Control Screen

- Select two crew members for a mission
- "Launch Mission" button
- Mission log displayed in a TextView or ScrollView
- Visual feedback for mission progress (health bars, threat status)

Statistics Screen (bonus)

- Per-crew-member stats: missions completed, wins, training sessions
- Colony-wide stats: total missions, total crew recruited, etc.

2 Basic Requirements

To successfully complete this project and meet the mandatory requirements, the following aspects must be implemented:

1. **Object-Oriented Programming (OOP) Principles**
   f. The program must be coded according to the object-oriented paradigm.
   g. Use appropriate classes (e.g., CrewMember, Storage, MissionControl, Threat).
   h. Follow encapsulation, inheritance, and polymorphism where necessary.

2. **Code Language**
   i. All code, comments, and documentation must be in English.

3. **Android App Development**
   j. The application must run on Android devices.
   k. It must be developed using Java in Android Studio.

4. **Basic Functionality**

The application must include the following features:

l. **Crew Management**
   i. Users can create different types of crew members (Pilot, Engineer, Medic, Scientist, Soldier).
   ii. Newly recruited crew members are placed in Quarters.
   iii. Users can move crew members to the Simulator or Mission Control.

m. **Training System**
   iv. Crew members gain experience points when trained in the Simulator.
   v. Experience increases skill power (e.g., if XP = 2, skill increases by 2 points).

n. **Cooperative Mission System**
   vi. Users can select two crew members to go on a mission.
   vii. A threat is generated by the system with stats based on a scaling formula.
   viii. Missions follow a turn-based system:

- Crew members take turns acting against the threat.
- The threat retaliates after each crew member's action.
- The mission continues until the threat's energy drops to zero or both crew members are defeated.
- Surviving crew members gain experience points.
- Defeated crew members are removed from the program.

o. **Crew Recovery**
   ix. When a crew member returns to Quarters, its energy is fully restored but experience points remain.

5. **Data Structures**
   p. The program must use data structures effectively.
   q. HashMap<Integer, CrewMember> could be used for storing crew members and their IDs.
   r. ArrayList may be useful for managing lists of crew members, especially with RecyclerView.

3 Grading Criteria

Implementing the mandatory features earns 13 points, and additional points can be gained by implementing extra features.

REMINDER: There are no resubmissions. This is your only attempt. Make sure everything works before submitting.

3.1 Mandatory Requirements (Required for 13 Points)

Requirement	Description	Points
Object-Oriented Code	The program is coded according to the object-oriented paradigm.	Required
Code in English	All code and comments must be in English. However, documentation and UI elements can be in Finnish.	Required
Android Compatibility	The program must work on various Android phones and be coded in Java using Android Studio.	Required
Basic Functionality	The program must implement all the features defined above so that users can interact with crew members.	Required
Documentation	The final submission must include documentation (PDF or GitHub .md file) containing: class diagram (excluding UI classes like Activities), division of work among team members, implemented features, and a general project description. NOTE: If ANY of these is missing, you will be deducted -8 points per missing section. If this would lower your grade to 0, we will floor it to 1. This way you can still pass, but your grade is just 1. If the GitHub link is missing or it is set as private so the teacher cannot access it, your project work cannot be evaluated. This will lead into failure of the project work.	Required

3.2 Bonus Features (Extra Points)

These points are added ONCE YOU HAVE A PASSING GRADE. Unless your project fulfils the minimum required features, these extra points do not matter. They only improve your grade, not help you get it from 0->1.

<u>Include the list of bonus features in your project report, otherwise these will not be graded!!!</u>

Feature	Description	Points
RecyclerView	Uses the RecyclerView component to list crew members and their details.	+1
Crew Images	Different crew specializations are visualized with unique images.	+1
Mission Visualization	By default, a textual description of the mission is displayed in a TextView. However, you can improve this by dynamically adding visual effects, such as crew member images, threat animations, or energy bar updates. Creativity is encouraged!	+2

Tactical Combat	The player controls the crew member's actions each turn (e.g., attack, defend, use special ability). The threat acts automatically based on simple AI logic.	+2
Statistics	Track how each crew member has performed (number of missions, victories, training sessions, etc.).	+1
No Death	Instead of dying, a defeated crew member is sent to Medbay and returns to initial stats after a penalty. Lost missions can also be tracked in statistics.	+1
Randomness in Missions	Add some randomness to missions, e.g., skill + Math.random() * 3.	+1
Specialization Bonuses	Certain specializations get bonuses on certain mission types (e.g., Engineer gets +2 skill on "repair station" missions). Teaches polymorphism effectively.	+2
Larger Squads	Allow 3-person squads on missions instead of just 2. Adds UI and logic complexity.	+2
Fragments	Use fragments meaningfully in the application.	+2
Data Storage & Loading	Save all crew members to a file and allow the user to load them when needed.	+2
Statistics Visualization	Visualize game statistics in some way (e.g., using AnyChart-Android).	+2
Custom Feature X	Implement and grade your own feature!	+0–2

By implementing all required features, the base score is 13 points. Additional features allow for more points, potentially increasing the final score significantly.

3.3 Penalty for Inappropriate Content

Issue	Description	Penalty
Inappropriate Content	If the program, documentation, comments, etc., contain inappropriate material (e.g., trash, hate speech, racism).	-5 to -100

4 Make Your Project More Meaningful?

Does this project feel uninspiring? Would you prefer to create a useful application for yourself?

Maybe your parents need an app for something?

If you have an idea for a different project, suggest it to the lecturer, and let's see if it can be turned into your coursework!